

Sistema de Semáforos Inteligentes

Trabajo Practico Integrador 2026

Paradigmas y Lenguajes de Programacion - Grupo 35

#Informe Técnico — Trabajo Práctico Integrador 2026

SISTEMA DE SEMÁFOROS INTELIGENTES

Ecosistema: Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas
Cátedra: Paradigmas de Programación

Grupo 35

INTEGRANTES:

- Romero, Matías
- Núñez, Axel
- Romero, Julieta
- Stoppello, Agustín
- Torres, Alexis



```
;; Scheme (RSRS)
(define (better-route? r1 r2)
  (< (route-cost r1)
    (route-cost r2)))

(define (update-signal state)
  (cond
    ((= (car state) 'red)
     (make-state 'green
                 (cadr state)))
    ((= (car state) 'green)
     (make-state 'yellow
                 (cadr state)))
    (else
     (make-state 'red
                 (cadr state)))))

(define (next-state state)
  (update-signal
   (evaluate-sensors state)))
```

ÁRBOL DE EVALUACIÓN SIMBÓLICA

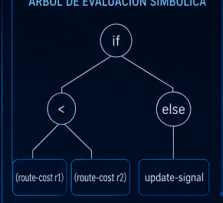
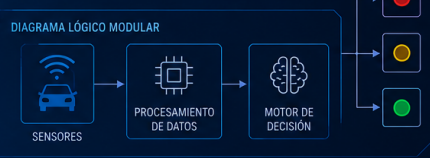


DIAGRAMA LÓGICO MODULAR



CIRCUITO LÓGICO



Universidad Nacional del Nordeste (UNNE) - FaCENA

Licenciatura en Sistemas de Informacion

1. Identificación del proyecto y equipo

El presente informe documenta el desarrollo del núcleo lógico de un sistema de semáforos inteligentes, implementado bajo restricciones propias del paradigma funcional: funciones puras, inmutabilidad, ausencia de bucles imperativos y paso explícito del estado mediante argumentos.

Integrante	Rol / responsabilidad	Usuario GitHub
Agustin M. Stoppello	Coordinación y organización general	@Agustinstpp
Matias Luciano Romero	Núcleo Common Lisp (Fase 1)	@matiasromero
Julieta Estefania Romero	Documentación e informe técnico	@Julieta-Git
Axel Gabriel Nunez	Integración de librerías (Fase 2)	@axeel-nunezz
Torres Alexis Joaquin	Lenguaje comparativo - Scheme (Fase 3)	@Torres-Alexis-Joaquin

2. Introducción y objetivos

El sistema modela el comportamiento de un semáforo urbano desde una perspectiva funcional. En lugar de modificar variables globales o estados internos, las funciones reciben los datos necesarios como parámetros y devuelven nuevos valores. Esto facilita la trazabilidad del comportamiento y reduce efectos secundarios no deseados.

- Implementar la lógica de transición de estados del semáforo.
- Calcular el color activo en un instante determinado mediante un temporizador.
- Recibir los tiempos del semáforo mediante una configuración pasada como argumento, manteniendo el flujo de datos sin variables globales mutables.
- Generar salidas de auditoría por pantalla y por archivo.
- Comparar la solución Common Lisp con una reimplementación en Scheme.
- Clasificar cada función según naturaleza, estrategia de control e impacto en memoria.

3. Fase 1 - Desarrollo del núcleo en Common Lisp

La primera fase implementa los requerimientos funcionales principales del sistema. Las funciones se escribieron evitando bucles imperativos y variables globales mutables. El estado del sistema, incluyendo la configuración de tiempos, fluye mediante parámetros.

transicion

- Naturaleza: Pura.
- Estrategia de control: Condicional.
- Impacto en memoria: No destructiva.

Modela el cambio de estado. Recibe el color actual y el color destino. Si el destino es válido, devuelve una lista con el estado y la acción literal correspondiente; si no, devuelve acción-por-defecto.

timer

- Naturaleza: Pura.
- Estrategia de control: Uso de mod y condicionales.
- Impacto en memoria: No destructiva.

Recibe un timestamp y una configuración. Calcula el instante dentro del ciclo total y devuelve el color activo.

mostrar-cambio

- Naturaleza: Impura.
- Estrategia de control: Salida por pantalla.
- Impacto en memoria: No destructiva.

Registra en terminal el cambio de una luz indicando tiempo, color anterior y color nuevo.

duracion-ciclo

- Naturaleza: Pura.
- Estrategia de control: Composicion aritmetica.
- Impacto en memoria: No destructiva.

Suma los tiempos obtenidos desde la configuracion externa para calcular la duracion total del ciclo.

recomendacion-ciclo

- Naturaleza: Pura.
- Estrategia de control: Condicional.
- Impacto en memoria: No destructiva.

Evalua si la duracion se encuentra dentro del rango recomendado de 35 a 150 segundos.

ciclos-por-tiempo

- Naturaleza: Pura.
- Estrategia de control: Aritmetica simple.
- Impacto en memoria: No destructiva.

Convierte minutos a segundos y calcula la cantidad de ciclos completos en ese periodo.

distribucion-hora

- Naturaleza: Pura.
- Estrategia de control: Calculo porcentual.
- Impacto en memoria: No destructiva.

Calcula el porcentaje de tiempo correspondiente a cada color segun la configuracion cargada.

informe

- Naturaleza: Impura.
- Estrategia de control: Recursividad local con labels y E/S.
- Impacto en memoria: No destructiva.

Genera un archivo de texto con el historial de eventos de auditoria.

4. Fase 2 - Integracion de Quicklisp y cl-json

En la segunda fase se incorporo una libreria externa mediante Quicklisp. La libreria seleccionada fue cl-json, utilizada para leer los tiempos de temporizacion desde un archivo JSON externo. Esto permite cambiar los parametros del sistema sin editar el codigo fuente.

Archivo de configuracion utilizado:

```
{  
  "rojo": 93,  
  "amarillo": 9,  
  "verde": 123  
}
```

La funcion cargar-config realiza la lectura del archivo y retorna una lista de asociacion. Como se trata de una operacion de entrada/salida, se clasifica como impura. En cambio, obtener-tiempo es pura: dado el mismo config y el mismo color, siempre retorna el mismo valor.

La decision mas importante fue evitar almacenar la configuracion en una variable global mutable. En su lugar, config se pasa como argumento a las funciones que lo necesitan. De este modo se respeta la inmutabilidad y el flujo explicito de datos exigido por el trabajo.

5. Funcion informe y persistencia de auditoria

La funcion informe complementa el registro por pantalla generando un archivo fisico llamado informe-ejecucion-semaforo.txt. Utiliza with-open-file para abrir el archivo y labels para definir funciones locales que recorren recursivamente la lista de logs.

- Naturaleza: impura, porque escribe en disco.
- Estrategia: recursividad local para procesar la lista de eventos.
- Impacto: no destructiva, porque no modifica la lista original de datos.

6. Fase 3 - Estudio comparativo: Scheme

Scheme es un dialecto de la familia Lisp, creado por Guy L. Steele y Gerald Jay Sussman. Se caracteriza por un diseno minimalista, sintaxis reducida y fuerte relacion con la programacion funcional. En el trabajo se utilizo para reimplementar las funciones de transicion y temporizacion.

Implementacion realizada

La funcion transicion mantiene la logica de cambio de estado mediante cond y el predicado eq?. La funcion timer recibe un timestamp y una configuracion externa, y calcula mediante modulo el color activo en un instante determinado. La configuracion se representa como una lista de asociacion con simbolos como rojo, verde y amarillo.

Comparacion sintactica

Common Lisp	Scheme / Racket
defun	define
eq	eq?
mod	modulo
T / NIL	#t / #f, segun el caso
Lisp-2	Lisp-1

Lisp-1 vs Lisp-2

Common Lisp es Lisp-2: funciones y variables viven en espacios de nombres separados. Por eso, al pasar funciones como valores puede requerirse el uso de #' y funcall. Scheme es Lisp-1: funciones y variables comparten un mismo espacio de nombres. Por esta razon, una funcion puede pasarse y ejecutarse como cualquier otro valor sin operadores adicionales.

Tail Call Optimization (TCO)

Scheme exige por especificacion la optimizacion de llamadas de cola. Si una llamada recursiva es la ultima accion de una funcion, el entorno puede reutilizar el marco actual de pila. Esto permite escribir procesos recursivos prolongados sin agotar memoria de pila.

```
(define (bucle-simulacion timestamp config)
  (bucle-simulacion (+ timestamp 1) config))
```

7. Bitacora de depuracion

Mutacion de estado por prototipado

En una version inicial se considero guardar config como variable global y modificarla. Esto fue descartado porque viola la inmutabilidad. La solucion fue retornar config desde cargar-config y pasarlo como argumento.

Diferencia de predicados en Scheme

Durante la migracion se reemplazo eq por eq?, ya que en Scheme los predicados suelen terminar con signo de pregunta y devuelven #t o #f.

Keywords de JSON y listas de asociacion

cl-json en Common Lisp produce claves como :rojo. En Scheme se trabajo con simbolos puros como rojo para que assoc funcione correctamente en la configuracion de prueba.

Nombres de funciones y convencion

Se normalizaron nombres segun la convencion solicitada: timer, duracion-ciclo, recomendacion-ciclo, ciclos-por-tiempo, distribucion-hora y mostrar-cambio.

8. Aseguramiento de calidad

El repositorio incluye ejemplos de uso para verificar el comportamiento normal, caminos alternativos y casos invalidos. Estos ejemplos permiten copiar y pegar llamadas en el REPL para validar transicion, timer, duracion-ciclo, recomendacion-ciclo, ciclos-por-tiempo, distribucion-hora e informe.

```
(transicion 'en-rojo 'verde)
(transicion 'en-rojo 'azul)
(timer 0 config)
(duracion-ciclo config)
(recomendacion-ciclo (duracion-ciclo config))
(ciclos-por-tiempo 15 config)
(distribucion-hora config)
```

9. Conclusiones

El desarrollo del sistema permitio aplicar conceptos centrales del paradigma funcional a un problema concreto. La restriccion de no utilizar estado global mutable obligo a que los datos circularan explicitamente por parametros, haciendo mas clara la relacion entre entrada y salida de cada funcion.

La integracion con cl-json demostro que una solucion funcional puede interactuar con configuracion externa sin abandonar la inmutabilidad del nucleo. Por otro lado, la migracion parcial a Scheme evidencio la cercania conceptual entre lenguajes de la familia Lisp y permitio comprender diferencias importantes como Lisp-1 frente a Lisp-2 y el uso de TCO.

10. Bibliografia

- Steele, G. L. y Sussman, G. J. (1975). Scheme: An Interpreter for Extended Lambda Calculus.
- Seibel, P. (2005). Practical Common Lisp. Apress.
- Racket Documentation. Standards R5RS y R6RS.
- Quicklisp Beta Release Documentation.
- Consigna Trabajo Practico Integrador 2026 - PyL - UNNE FaCENA.